Homework 1: 3D Scene Reconstruction

Human-centered Intelligent System Lab
National Yang Ming Chiao Tung University

Fall 2026

1 Introduction

Imagine asking a household robot to bring you something from the kitchen. To do that safely, it needs a useful picture of the home: where rooms and furniture are, how spaces connect, and where it can move without collisions. A 3D scene map provides a spatial representation of the environment that can support navigation and scene understanding [1].

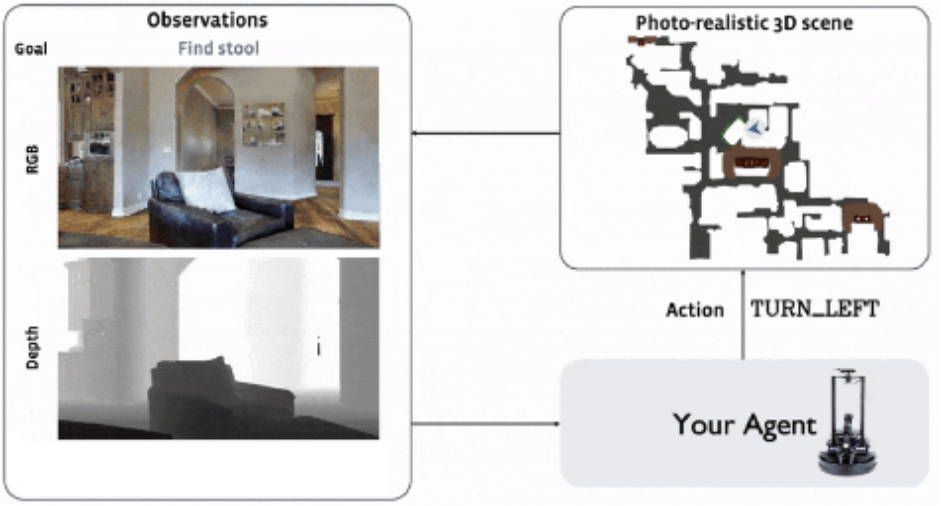


Figure 1: Visual pipeline from RGB-D observations to a 3D scene.

In this assignment, you will reconstruct apartment_0, an indoor scene from the Replica dataset [2], using RGB-D (color and depth) observations in Habitat-Sim [3] simulator. The reconstruction pipeline converts depth images into 3D point clouds, registers observations from multiple viewpoints, integrates them into a unified 3D scene, and evaluates the result through visual inspection and comparison of the estimated camera trajectory with ground truth. In addition to reconstruction, you will examine the input observations to understand how conditions such as missing or noisy depth, insufficient viewpoint overlap, and large frame-to-frame motion relate to the resulting map and trajectory. Together, these components connect the characteristics of the input data with the behavior of the reconstruction process. Although conducted in simulation, this assignment provides a starting point for considering the challenges involved in building and using 3D scene maps for real-world robotic tasks.

By completing this assignment, you will be able to:

1. Understand the pipeline from RGB-D observations to 3D scene reconstruction, including the pinhole camera model, point-cloud generation, registration, mapping, and trajectory estimation.
2. Analyze input observations and capture choices to identify factors associated with reconstruction behavior and failure cases.

3. Evaluate and communicate reconstruction results using quantitative metrics, visualizations, and data-verification evidence.

2 Structure and Workflow

This assignment is organized into **two phases** that share the same reconstruction and data-verification workflow. The main difference lies in how the input observations are obtained:

- **Phase 1:** Use a provided first-floor batch containing designed data-quality variations to reconstruct a 3D scene map, diagnose factors that contribute to reconstruction failures, and refine the reconstruction based on the analysis.
- **Phase 2:** Extend the same workflow to the second floor, where you will collect your own RGB-D observations and camera poses through navigation in the simulation and reconstruct the scene from the collected data.

For the reconstruction component, the workflow follows the general registration procedure described in the **Open3D** [4] documentation, including point-cloud downsampling, global registration, and local refinement. The results from both phases are then compared to examine how data-collection choices and observation characteristics relate to reconstruction outcomes.

2.1 3D Scene Map Reconstruction

2.1.1 Reconstruction Workflow

The reconstruction work in both phases follows the pipeline below:

1. **Depth Unprojection.** Convert valid depth pixels to 3D points using the pinhole camera model. Use the resolution, field of view (FOV), and depth scale recorded in `intrinsics.json`. Implement this mapping yourself; **do not use Open3D projection functions for this step.**
2. **Voxelization (Downsampling).** Reduce the number of points to improve computational efficiency while retaining the scene structure. Open3D functions may be used for this step[4].
3. **Global Registration (RANSAC).** Estimate an initial alignment between point clouds to handle large viewpoint differences using the Random Sample Consensus (RANSAC) [5]. Open3D functions may be used for this step [4].
4. **Local Registration (ICP).** Starting from the global-registration result, refine the transformation between point clouds using the Iterative Closest Point (ICP) [6] algorithm. Open3D's point-to-point or point-to-plane ICP implementation may be used for this step [4].
5. **Scene Map Reconstruction and Evaluation.** Transform and accumulate the aligned point clouds into a unified 3D scene map and construct the estimated camera trajectory. For clearer visualization, remove the ceiling when necessary, and visualize the estimated trajectory in red and the ground-truth trajectory in black. Evaluate the estimated trajectory against the ground truth using the required Mean L2 Distance. Open3D functions may be used for point-cloud processing, transformation, and visualization [4].

After completing the reconstruction, evaluate the resulting scene and camera trajectory together with the **data-verification process**. If the reconstruction contains noticeable artifacts or alignment failures, identify likely contributing factors from the observations and reconstruction settings, adjust the relevant preprocessing or registration parameters, and rerun the pipeline as needed.

2.1.2 Depth Unprojection

Each RGB-D observation contains a color image and a depth image corresponding to the same camera viewpoint. For reconstruction, each valid depth pixel is converted into a 3D point in the camera coordinate

frame using the pinhole camera model. Given calibrated camera intrinsics (f_x, f_y, c_x, c_y) and a valid stored depth value $D(u, v)$ at pixel (u, v), the metric depth is first computed as $Z = D(u, v)/s$, where s is the depth scale. The corresponding 3D point is then obtained by back-projecting the pixel into the camera coordinate frame [7]:

$$X = \frac{(u - c_x)Z}{f_x}, \qquad Y = \frac{(v - c_y)Z}{f_y}, \qquad Z = \frac{D(u, v)}{s}.$$

The symbols in this equation are annotated as follows:

X, Y, Z :	3D coordinates in the camera frame,	u, v :	pixel column and row,
f_x, f_y :	horizontal and vertical focal lengths (pixels),	c_x, c_y :	principal-point coordinates (pixels),
$D(u, v)$:	stored depth value,	s :	scale from raw depth to meters.

Applying this unprojection to all valid depth pixels produces a point cloud for each RGB-D observation. These point clouds form the geometric input to the registration stage.

2.1.3 Global and Local Registration

Each point cloud is expressed in its own camera coordinate frame, so observations from different viewpoints cannot be directly combined. Registration estimates the relative transformations needed to align them in a shared coordinate system. Following the steps in Open3D document [4], the alignment is then performed in two stages: global registration with Random Sample Consensus (RANSAC) [5] algorithm, and local refinement with Iterative Closest Point (ICP) [6] algorithm.

Global Registration. Global registration estimates an initial alignment between point clouds without requiring a close initial pose. The point clouds are first downsampled and represented using Fast Point Feature Histograms (FPFH) [8], which provide local geometric descriptors for establishing candidate correspondences. Random Sample Consensus (RANSAC) [5] then repeatedly samples candidate correspondences, estimates a transformation, and rejects inconsistent matches using geometric consistency checks. Only correspondences that pass these checks are used to estimate and validate the transformation. The resulting alignment provides a coarse global registration that can be further refined in the local-registration stage.

Local Registration. Local registration further refines the alignment between point clouds using Iterative Closest Point (ICP) [6]. Starting from an initial transformation, ICP repeatedly establishes correspondences between the source and target point clouds, estimates a transformation that reduces the alignment error, and updates the source point cloud. This process continues until the convergence criterion is reached. Open3D provides both point-to-point and point-to-plane formulations for ICP [4]. Point-to-point ICP minimizes the Euclidean distance between corresponding points, whereas point-to-plane ICP minimizes the distance from each source point to the tangent plane defined by the corresponding target point and its surface normal.

After global and local registration, the estimated pairwise transformations describe the relative poses between point clouds. By choosing one point cloud as the reference frame and composing these transformations, the remaining point clouds can be transformed into the same coordinate system and accumulated to form a complete scene map. Registration performance depends on parameters such as voxel size, correspondence thresholds, and convergence criteria. Different settings may affect alignment quality, robustness, runtime, and ultimately the reconstructed scene. These parameter choices should therefore be examined and documented during the reconstruction process.

2.1.4 Scene Map Reconstruction and Evaluation

After registration, the aligned point clouds are accumulated into a reconstructed 3D scene, while the estimated pairwise transformations are composed to recover the camera trajectory. Reconstruction is

evaluated through both visual inspection of the scene map and quantitative comparison of the estimated trajectory with the ground-truth (GT) trajectory. GT poses are used only for evaluation and are not used as inputs to the reconstruction process.

For quantitative trajectory evaluation, this assignment uses the **Mean L2 Distance**, defined as the mean Euclidean L_2 distance between estimated and ground truth camera positions. For N evaluated frames, use:

$$\text{MeanL2} = \frac{1}{N} \sum_{i=1}^{N} \left\| \hat{\mathbf{p}}_i - \mathbf{p}_i^{\text{GT}} \right\|_2 ,$$

The symbols in this equation are annotated as follows:

$$\begin{aligned} N &: \text{ number of evaluated frames}, \quad & i &: \text{ frame index}, \\ \hat{\mathbf{p}}_i = (\hat{x}_i, \hat{y}_i, \hat{z}_i) &: \text{ estimated camera position}, & & \\ \mathbf{p}_i^{\text{GT}} = (x_i^{\text{GT}}, y_i^{\text{GT}}, z_i^{\text{GT}}) &: \text{ ground-truth camera position}, & & \\ \|\cdot\|_2 &: \text{ Euclidean } L_2 \text{ norm}. & & \end{aligned}$$

For each capture, the first camera pose defines the reference frame. The estimated trajectory is initialized in this frame, and the GT trajectory is transformed to the same frame for evaluation. This common reference frame is the shared scene coordinate system. A lower Mean L2 Distance indicates closer agreement between the estimated and ground-truth camera poses.

2.2 Data Verification

In Physical AI, a robot acts from time-varying sensor observations, so the quality of those observations can directly affect downstream perception and especially in reconstruction. Missing depth, noisy measurements, insufficient geometric support, or abrupt camera motion may lead to alignment failures even when the reconstruction method itself is unchanged. Data verification therefore helps identify and document observation conditions before interpreting reconstruction results.

Raw images and numerical measurements do not, by themselves, describe how a data-quality observation relates to an experiment or a reconstruction run. This assignment uses a **semantic layer** to record these relationships explicitly. An **ontology** [9] defines the shared concepts and relationships used to describe capture batches, frames, quality factors, measurements, verification results, and reconstruction runs.

2.2.1 Data Verification Workflow

The data-verification workflow connects the conditions observed in the input data with the reconstruction results. It consists of the following stages:

- **Experiment Declaration.** Declare an experiment for a capture batch and specify any reconstruction-setting adjustments. At this stage, no quality factors or qualification thresholds are selected.
- **First-Pass Reconstruction.** Run reconstruction on the declared capture while measuring all predefined data-quality factors. These measurements provide an initial description of the conditions observed throughout the sequence.
- **Factor Recording.** Record the raw factor measurements as experiment-scoped evidence. Measurements are associated with individual frames or consecutive frame pairs, depending on the factor definition. No qualification status or Pass/Fail decision is assigned.
- **Data Inspection.** Inspect the recorded measurements through summaries and visualizations of their temporal behavior, distributions, and relationships. This allows the student to identify notable regions, patterns, and possible tradeoffs in the captured data.
- **Data Intervention and Reconstruction.** Based on the observed evidence, formulate an intervention intended to improve or probe reconstruction behavior. Data selection is one possible intervention,

where observations are retained or excluded according to discovered factor patterns. Other interventions may modify how the available data is used while preserving the recorded measurements as unchanged evidence. Reconstruction is then repeated to evaluate the effect of the intervention.

- **Experiment Reporting.** Treat the sequence of experiment declarations, recorded factor measurements, interventions, and reconstruction outcomes as a structured lab notebook. These records provide the evidence used to explain and compare reconstruction behavior across experiments.

The purpose of this workflow is not to classify observations as inherently good or bad before reconstruction. Instead, it first records the observable data conditions, then uses those observations to motivate and evaluate interventions. Data selection is therefore one possible response to the evidence rather than the objective of the workflow itself. Different interventions can be compared using the same recorded measurements and reconstruction procedure.

2.2.2 Data Quality Factors

The available factors are **pipeline-grounded measurements**: each factor records a first-order quantity that is already produced or directly observed during the reconstruction process. Rather than introducing separate image-quality heuristics, the factors expose intermediate states from point-cloud generation, voxel downsampling, coarse registration, and local refinement.

The measurements are applied either to a single depth image or to an ordered pair of consecutive depth images.

- **Single-depth-image factors**
  - `RawPointCount`: the number of valid back-projected 3D points obtained from depth pixels with positive depth values before voxel downsampling.
  - `VoxelizedPointYield`: the number of points remaining after the reconstruction pipeline applies voxel downsampling. It describes how much geometric information survives the reduction step.
- **Consecutive depth-pair factors**
  - `CoarseRANSACFitness`: the correspondence inlier fraction reported by the coarse RANSAC registration stage.
  - `CoarseRANSACRMSE`: the root-mean-square error of the inlier correspondences produced by coarse RANSAC registration.
  - `CoarseRANSACCorrespondences`: the number of correspondences retained in the RANSAC registration result.
  - `ICPInlierFitness`: the correspondence inlier fraction reported by the ICP refinement stage.
  - `ICPInlierRMSE`: the root-mean-square error of the inlier correspondences after ICP refinement.
  - `ICPInlierCorrespondences`: the number of correspondences retained in the ICP registration result.

These factors are diagnostic measurements of the reconstruction pipeline. Together, they expose how much geometric support is available and how strongly consecutive observations agree during coarse and local alignment. Students can inspect these measurements alongside reconstruction outcomes to identify patterns, formulate hypotheses, and motivate subsequent data interventions.

3 Implementation Details

This section provides practical instructions for completing the assignment, including environment setup, phase-specific data preparation and collection, and reconstruction execution. The provided code in **utils.py** templates mark all required implementation sections with `TODO`.

3.1 Environment Setup

3.1.1 Pixi installation

Pixi manages the repository's software environment. Install it using the official instructions for your operating system, then restart the shell so its executable is available on the system path [10].

```
curl -fsSL https://pixi.sh/install.sh | sh
# Or use the official package-manager instructions for your platform.
```

3.1.2 Repository setup

The main support target is 64-bit Linux. Habitat-Sim and the course workflow are configured and verified primarily on Linux. macOS and Windows with WSL may be used for development, but platform-specific graphics or binary differences may require additional adjustments.

Linux. Clone the course repository and install its Habitat environment from the repository root. The data-verification workflow uses RDFLib for working with RDF/Turtle experiment records and SPARQL queries [11]:

```
git clone --recurse-submodules https://github.com/HCIS-Lab/physicalai26-hw.git
cd physicalai26-hw/hw1
pixi install -e habitat
pixi run -e habitat python -c "import open3d, rdflib; print('ready')"
```

macOS. Install Pixi using the official macOS guide [10]. Clone the repository as above and attempt `pixi install -e habitat` using the Habitat-Sim installation guidance [12]. Linux remains the primary platform; if simulator binaries or visualization dependencies are unavailable, use Linux or WSL.

Windows with WSL. Install WSL2 with Ubuntu, then run Linux commands inside the WSL terminal. Keep the repository and datasets within the WSL Linux filesystem. Native Windows is not the main supported target [13].

3.2 Experiment, Measurement, and Reconstruction Workflow

The workflow declares an experiment over a capture, records pipeline-derived factor evidence during a first-pass reconstruction, inspects the resulting measurements through tables and plots, and uses the observed evidence to formulate and evaluate data-intervention strategies. These interventions may include data selection or other changes to how the captured observations are used in reconstruction.

3.2.1 Declare an experiment

First create a declaration for the capture. This creates an experiment workspace containing:

- `experiments/<experiment-name>/`: the workspace for the experiment.
- `experiments/<experiment-name>/plots/`: generated plots for inspecting recorded factors.
- `experiments/<experiment-name>/<experiment>.ttl`: the experiment declaration and recorded evidence.
- `experiments/<experiment-name>/queries/`: SPARQL queries used for subsequent analysis and interventions.

```
pixi run -e habitat python api.py declare
--name <experiment-name>
--data-dir <CAPTURE>
--floor 1
```

3.2.2 First-Pass Reconstruction

After creating the experiment workspace, run a first-pass reconstruction over the full capture:

```
pixi run -e habitat python reconstruct.py
  --experiment experiments/<experiment-name>
```

During this pass, the reconstruction pipeline automatically records the eight pipeline-grounded factors introduced above. `RawPointCount` and `VoxelizedPointYield` are recorded for each depth image, while the RANSAC- and ICP-based factors are recorded for each adjacent depth pair. These measurements are appended to the previously declared experiment Turtle file.

3.2.3 Inspect, Plot, and Evaluate

After the first-pass reconstruction, inspect the recorded factor measurements together with the reconstruction result. The reconstruction uses the capture's camera intrinsics, performs back-projection, voxel downsampling, coarse RANSAC alignment, and ICP refinement.

The experiment can then be explored through terminal summaries and generated plots:

```
pixi run -e habitat python api.py explore
  experiments/<experiment-name>

pixi run -e habitat python api.py explore
  experiments/<experiment-name> --launch-dashboard
```

The generated plots are stored under `experiments/<experiment-name>/plots/` and summarize factor timelines, distributions, and relationships with reconstruction outcomes. Since the recorded factors contain raw measurements rather than predefined Pass/Fail statuses, students should use these visualizations to identify meaningful patterns or value ranges.

Report the reconstruction result together with the relevant settings and evaluation metrics, including trajectory error, evaluated frame count, retained-frame fraction, and runtime. Use the recorded factor measurements to explain observed alignment failures, trajectory errors, or map artifacts, and to motivate subsequent data interventions.

3.2.4 Apply Data Interventions and Reconstruct

After inspecting the recorded factors, use the observed evidence to formulate a data intervention and evaluate its effect through reconstruction. Frame selection with SPARQL is one intervention strategy: the query can retain or exclude observations according to factor values, frame indices, or patterns identified during inspection.

For example, the following query selects frames according to observed `VoxelizedPointYield` and `ICPInlierFitness` ranges:

```
PREFIX hw1: <http://taica.course/hw1/ontology#>
PREFIX exp: <http://taica.course/hw1/experiment/>

SELECT DISTINCT ?frameIndex WHERE {
  exp:<experiment-name> hw1:onBatch ?batch .
  ?batch hw1:hasFrame ?frame .
  ?frame hw1:frameIndex ?frameIndex ;
         hw1:hasDepthImage ?depth .

  FILTER NOT EXISTS {
    ?sOcc a hw1:SingleImageFactor ;
          hw1:inExperiment exp:<experiment-name> ;
          hw1:hasCurrentFrame ?depth ;
          hw1:value ?sv ;
```

```
          hw1:hasDefinition hw1:VoxelizedPointYield .
    FILTER (?sv > 50000)
  }

  FILTER NOT EXISTS {
    ?pOcc a hw1:DepthPairFactor ;
          hw1:inExperiment exp:<experiment-name> ;
          hw1:hasCurrentFrame ?depth ;
          hw1:value ?pv ;
          hw1:hasDefinition hw1:ICPInlierFitness .
    FILTER (?pv < 0.05)
  }
}
ORDER BY ?frameIndex
```

Inspect the query result before using it in reconstruction:

```
pixi run -e habitat python api.py query
  experiments/<experiment-name>
  --query-file experiments/<experiment-name>/queries/frames.rq

pixi run -e habitat python reconstruct.py
  --experiment experiments/<experiment-name>
  --selection-query experiments/<experiment-name>/queries/frames.rq
  --no-vis
```

The resulting reconstruction can then be compared with the first-pass result to evaluate whether the intervention changes reconstruction behavior. Frame selection is only one possible intervention; the same evidence-driven process can be used to motivate other changes to how the captured observations are used during reconstruction.

3.3 Phase 1: First Floor Reconstruction

Phase 1 applies the workflow introduced above to the provided first-floor capture. Download the Phase 1 dataset archive and unpack it under `eval/<DATA_DIR>` without changing the provided directory structure.

Begin by declaring an experiment for the capture and running the first-pass reconstruction. Inspect the recorded pipeline-grounded factors together with the reconstruction result to identify patterns that may explain alignment errors, trajectory drift, or map artifacts.

Based on this evidence, formulate one or more data interventions and reconstruct the capture again to evaluate their effects. Compare the resulting runs with the first-pass reconstruction and justify each intervention using the observed factor measurements and reconstruction outcomes.



Figure 2: Reference first-floor reconstruction results (data size 152). The red and black lines show the estimated and ground-truth trajectories.

3.4 Phase 2: Second Floor Reconstruction

Phase 2 applies the same workflow to a capture collected by the student on the second floor of `apartment_0`. From `physicalai26-hw/hw1`, run:

```
pixi run -e habitat python load.py \
    --config configs/second_floor.yaml \
    --output-root eval/_data/second_floor/<student-id>
```

Use `w`/`s` to move, `a`/`d` to turn, `c` or Space to capture a frame, and `q` or Escape to finish. Record the capture path, route, and frame count.

After collection, declare an experiment and run the first-pass reconstruction. Inspect the recorded pipeline-grounded factors together with the reconstruction result, then formulate and evaluate data interventions using the same process as in Phase 1.

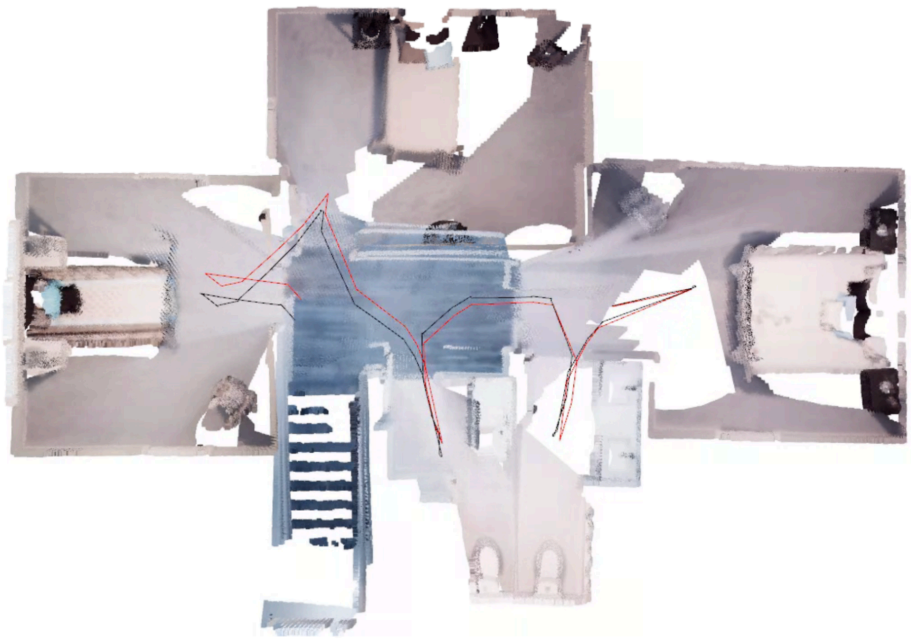


Figure 3: Reference second-floor reconstruction result (data size 213). The red and black lines show the estimated and ground-truth trajectories.

3.5 Bonus Track: Custom ICP Implementation

Implement `my_local_icp_algorithm` with nearest-neighbor correspondence search and rigid transformation estimation. Compare its reconstruction results and runtime with the Open3D-based ICP implementation to earn bonus points. The optional bonus is separate from the base score.

4 Grading

The following criteria describe the evaluation considerations for the required implementation, report, and optional bonus. Specific grading policies are determined by each partner institution.

4.1 Assessment Rubric

The submitted work is assessed based on implementation correctness, data integrity, reconstruction quality, reproducibility, and the quality of the report and discussions. Automated evaluation scripts may be used to verify required functionality and reproducibility.

- **Program Implementation:**
  - **Data Integrity:** The submitted workflow must produce valid experiment Turtle records: a well-formed declaration, a sealed machine section with the complete set of pipeline-grounded factor occurrences, and recorded baseline and selected reconstruction runs. Invalid Turtle syntax, a broken seal, missing factor occurrences, or missing run records lose credit.
  - **Code Implementation:** All required `TODO` blocks must be implemented correctly: depth unprojection from the capture's own intrinsics, voxel downsampling with normals and FPFH features, Open3D point-to-plane ICP refinement, trajectory composition and Mean L2 evaluation with red estimated and black ground-truth trajectories, and the full frame-to-frame reconstruction pipeline. Depth unprojection must be implemented from scratch without directly relying on Open3D's built-in functions. Incomplete, incorrect, or non-reproducible implementations lose credit.

  - **Reconstruction Quality:** Reconstruction quality is evaluated based on two complementary aspects:
    * **Camera Trajectory Agreement:** The Mean L2 Distance measures the positional difference between the estimated and ground-truth camera trajectories. A lower value indicates closer agreement with the ground truth.
    * **Scene Coverage and Completeness:** The reconstructed map should sufficiently cover the target floor, including major rooms, connecting spaces, and relevant corners, rather than only a limited portion of the environment.

- **Report:** The report must present a comparison table for the reconstruction runs, including Mean L2 Distance, frame count, total execution time, and the hyperparameters tested. It should discuss the results and findings, supported by:
  - Screenshots of the `apartment_0` Floor 1 and Floor 2 reconstructions, with ceilings removed when needed, estimated trajectories in red, and ground-truth trajectories in black.
  - Screenshots of the generated verification plots.
  - Explanations of the approaches used in both phases to improve reconstruction quality, supported by data-verification evidence such as observed factor patterns, frame-selection criteria or queries, and comparisons of reconstruction results before and after adjustments.

Bonus Track: Custom ICP Implementation. Students who implement a custom ICP algorithm must explain their implementation approach, compare its reconstruction results and execution time with the Open3D implementation, and discuss any techniques used to improve performance, such as correspondence search, robust weighting, threshold selection, convergence criteria, or runtime-accuracy tradeoffs. Incomplete explanations or missing comparisons will result in partial credit.

4.2 Report Format

The report should be submitted as a PDF, written in English, and kept concise. It should include the following content:

- **Implementation**
  - **Reconstruction Pipeline**
    * Explain the implementation of each major reconstruction step.
    * Describe the settings and parameter choices used in the pipeline.
    * Discuss implementation issues encountered, the solutions used to address them, and the rationale behind those choices.
  - **Data Verification**
    * Report the required data-quality measurements, measurement settings, and source links.
    * Identify relevant failure cases and explain how the observed evidence relates to reconstruction artifacts or alignment failures.
    * Describe any preprocessing or reconstruction adjustments made based on the analysis.
- **Results and Discussion**
  - Present reconstruction results for both phases.
  - Provide reconstruction screenshots with the ceiling removed, the estimated trajectory in red, and the ground-truth trajectory in black.
  - Provide a comparison table for the reconstructions being compared, including:
    * Mean L2 Distance;
    * Frame count;
    * Total execution time;
    * Tested hyperparameters.

  - Compare the two phases and discuss the main findings based on both reconstruction results and data-verification evidence.

- **Questions**
  1. What happens if ICP is performed without global registration (RANSAC)? Why?
  2. Describe the choices used to improve ICP stability, such as voxel size, correspondence thresholds, or robust losses.
  3. How do the observed data-quality factors and collection choices relate to reconstruction performance and Mean L2 Distance?

5 Submission

5.1 Submission Package

Please submit `<student-id>_hw1.zip` to the designated submission platform. Please use the following structure at the zip root and avoid adding an extra enclosing directory:

```
<student-id>_hw1.zip
|-- load.py
|-- reconstruct.py
|-- utils.py
|-- api.py
|-- ontology/
|   `-- hw1.ttl
|-- report.pdf
`-- README.md
```

- Include all source files used for the reconstruction and data-verification workflow.
- If the optional bonus track is attempted, include `my_local_icp_algorithm` in the submitted source and identify it in `README.md`. The bonus implementation should not replace the required Open3D-based pipeline.
- Do not include experiment Turtle files, generated diagnostics, Phase 2 capture data, Replica environment assets, or unrelated build artifacts.

5.2 Submission Rules

Submission deadlines and specific penalty policies vary by institution. Please follow the announcements and designated submission platform provided by your institution.

- **Deadline:** Submit the assignment by the deadline announced by your institution.
- **Late Submission:** Late submissions will be subject to a penalty. The specific deduction follows your institution's policy.
- **Submission Format:** Please follow the required file naming, directory structure, file paths, and report format. Incorrect submission formats, including a report not written in English, may result in a penalty according to your institution's policy.
- **Plagiarism:** All submitted work must be original. Plagiarism, including unauthorized copying of code or report content from other students or public sources, is prohibited and will be handled according to your institution's policy.

References

[1] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022. DOI: 10.1007/978-3-030-34372-9

[2] J. Straub et al., *The replica dataset: A digital replica of indoor spaces*, 2019. arXiv: 1906.05797. [Online]. Available: https://arxiv.org/abs/1906.05797

[3] M. Savva et al., "Habitat: A platform for embodied ai research," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019. arXiv: 1904.01201. [Online]. Available: https://arxiv.org/abs/1904.01201

[4] Open3D contributors, *Global registration*, Open3D Documentation, version 0.19.0, 2024. Accessed: Sep. 26, 2026. [Online]. Available: https://www.open3d.org/docs/0.19.0/tutorial/pipelines/global_registration.html

[5] M. A. Fischler and R. C. Bolles, "Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981. DOI: 10.1145/358669.358692

[6] Y. Chen and G. Medioni, "Object modelling by registration of multiple range images," in *Proceedings of the IEEE International Conference on Robotics and Automation*, 1992, pp. 2724–2729. DOI: 10.1109/ROBOT.1991.132043

[7] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.

[8] R. B. Rusu, N. Blodow, and M. Beetz, "Fast point feature histograms (fpfh) for 3d registration," in *2009 IEEE international conference on robotics and automation*, IEEE, 2009, pp. 3212–3217.

[9] N. Guarino, D. Oberle, and S. Staab, "What is an ontology?" In *Handbook on Ontologies*, S. Staab and R. Studer, Eds., Springer, 2009, pp. 1–17.

[10] Prefix.dev, *Pixi installation*. Accessed: Sep. 26, 2026. [Online]. Available: https://pixi.prefix.dev/latest/installation/

[11] RDFLib contributors, *Rdflib documentation*, RDFLib Documentation. Accessed: Oct. 7, 2026. [Online]. Available: https://rdflib.readthedocs.io/en/stable/

[12] Habitat-Sim contributors, *Habitat-sim installation*. Accessed: Sep. 26, 2026. [Online]. Available: https://github.com/facebookresearch/habitat-sim/blob/main/README.md

[13] Microsoft, *Install wsl*. Accessed: Sep. 26, 2026. [Online]. Available: https://learn.microsoft.com/windows/wsl/install